

7. Zeit

Michael Schöttner

Grundlagen Verteilter Systeme

SS 2026

7.1 Vorschau

- Reale Zeit
- Uhrensynchronisierung
- Lamport-Zeit
- Vektor-Zeit

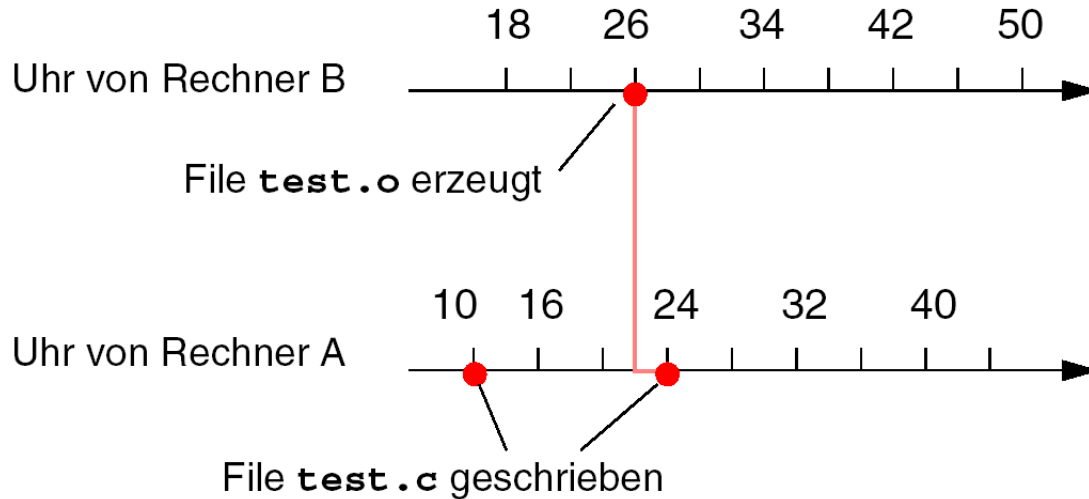
5.2 Reale Zeit

Motivation für synchrone Uhren

- Planung übergreifender Programmabläufe
- Geordnete Updates und Multicasts
- Gültigkeit von Berechtigungen
- Navigation
- ...

Beispiel: UNIX make (verteilter Fall)

- Editor läuft auf Rechner A, Compiler läuft auf Rechner B
- Problem:
 - Quelldatei ist augenscheinlich älter als die Zieldatei
 - Wird `make` aufgerufen, so wird sie nicht neu kompiliert



Technik der Zeitmessung - Sonnensekunde

- Beobachtung: Jeden Mittag erreicht die Sonne ihren höchsten Punkt am Himmel
- Die Zeitspanne zwischen zwei aufeinanderfolgenden Ereignissen dieses Typs heißt Tag, genauer gesagt ein Sonnentag
- Eine Sonnensekunde ist $1 / 86400$ dieser Zeitspanne
- **Aber eine Sonnensekunde ist nicht konstant**
→ wird aufgrund der Erddrehung immer länger

Technik der Zeitmessung - Atomuhr

- 1948: Bau der ersten Atomuhr
- 1 Sekunde ist die Zeit, die ein Cäsium-133-Atom benötigt, um 9.192.631.770 Mal zu schwingen. Die Schwingung ist sehr konstant.
- Am 1.1.1958 (Atomzeiteinführung) entsprach dies genau einer Sonnensekunde.



https://www.meinberg.de/german/info/atomic_clock.htm

Technik der Zeitmessung – Internationale Atomzeit

- Temps Atomique International (TAI) → Bureau International de l'Heure (BIH) in Paris ermittelt den Durchschnitt aus 260 Atomuhren (weltweit verteilt)
- TAI ist sehr genau und eine Sekunde der Atomzeit ist konstant
- Foto: BIH im Pariser Observatorium



<https://www.revolvy.com/page/Paris-Observatory>

Technik der Zeitmessung – Koordinierte Weltzeit

- = Universal Coordinated Time (UTC)
- Problem: TAI sehr genau und konstant, aber der Sonnentag wird immer länger
 - Damit würde der Mittag immer früher und früher liegen
- Lösung: **Schaltsekunden** einfügen, falls Diskrepanz $\geq 800\text{ms}$
 - Diese werden über das Internet angekündigt
- In 2015 entsprach die Differenz 36 Sekunden gegenüber dem Jahr 1958

Technik der Zeitmessung – Bemerkungen

- **Greenwich Mean Time (GMT)**
 - Mittlere Sonnenzeit am Nullmeridian (Zeit abhängig vom Längengrad und damit wann dort der „Mittag“ beobachtet wird)
 - GMT wurde abgelöst durch UTC
- **Central European Time (CET) = Mitteleuropäische Zeit (MEZ)**
 - Mittlere Sonnenzeit des 15. Längengrads östlich von Greenwich → UTC+1h
- **Central European Summer Time (CEST)**
= Mitteleuropäische Sommerzeit (MEZ)
 - UTC+2h

Quellen realer Zeit – Zeitgeber bei x86 CPUs

- Rechnerinterner Quarz für **Zeitgeber** / Systemzeit
 - 8253 Zeitgeber: 1,19 MHz (für alte PCs; ist nicht die BIOS-Uhr)
- **Time-Stamp-Counter**: inkrementiert bei jedem Takt
 - Achtung: moderne CPUs variieren Takt
 - Es gibt aber Time-Stamp-Counter die unabhängig vom aktuellen Takt mit konstanter Geschwindigkeit hochgezählt werden
- **APIC-Timer**: läuft mit Takt vom System-Bus (z.B. 800 MHz)
- **HPET**: High Precision Event Timer (mind. 10 MHz, bis zu 32 Counter)

Zugang zu Zeit von Atomuhren

- Zeitzeichen über Rundfunk und TV
- GPS-Satelliten:
 - Atomuhr mit ca. $0.1\mu\text{sec}$ Auflösung
 - notwendig für genaue Ortsbestimmung
- DCF77 Signal aus Mainflingen bei Frankfurt:
 - 77,5 kHz, 50 kW, Radius ca. 2.000 km
 - Ausstrahlung eines Zeitsignals basierend auf Atomuhr der Physikalisch-Technischen Bundesanstalt in Braunschweig

Physikalische und elektronische Grenzen

- Wunsch: Gleichlauf der Uhren in einem verteilten System
- Aber: Unterschiedliche Uhren laufen auch unterschiedlich schnell!
 - **Abweichung (engl. skew)**: Differenz zweier Uhren zu einem bestimmten Zeitpunkt.
 - **Drift**: Die Frequenz ist minimal unterschiedlich (auch bei Uhren desselben Typs!) und hängt zudem von Umgebungsbedingungen (u.a. der Temperatur) ab.

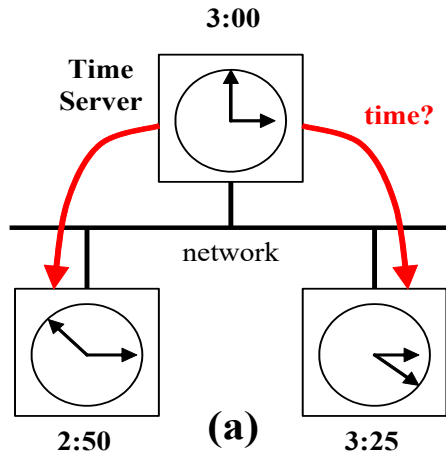
5.3 Uhren-Synchronisierung

Synchronisierung realer Uhren - Berkeley Verfahren

- Entwickelt für Gruppe von BSD-UNIX Rechnern im **LAN** (1989)
 - Falls kein UTC-Empfänger zur Verfügung steht, Uhr des Servers manuell korrigieren
 - Genauigkeit im LAN cirka 10-20ms
- Ein Rechner ist der **aktive Koordinator (=Time-Server)**
 - a) Frägt bei allen anderen Rechnern nach deren aktuellen Zeit
 - b) Errechnet aus empfangenen Zeiten den Durchschnitt aller Abweichungen
 - c) Jeder Knoten erhält individuelle Korrekturwerte
- Bem.: Klienten mit zu großer Abweichung bei der Mittelung nicht berücksichtigen

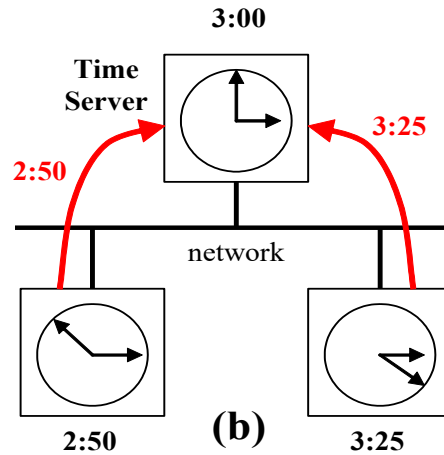
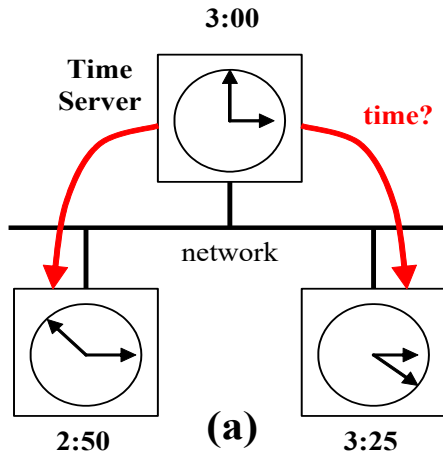
Berkeley Verfahren - Beispiel

a) Server erfragt Zeit bei allen Clients



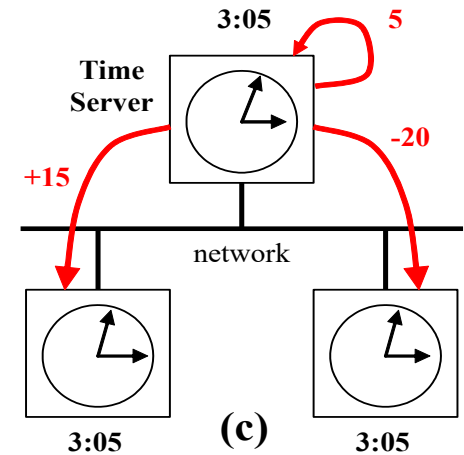
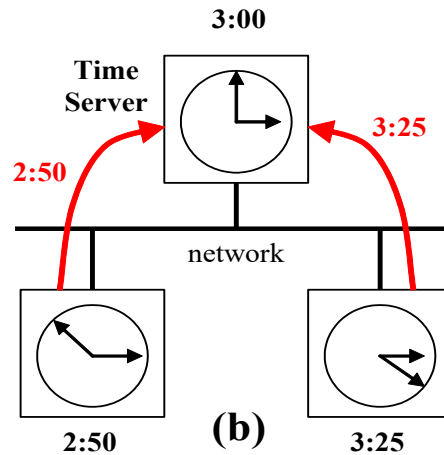
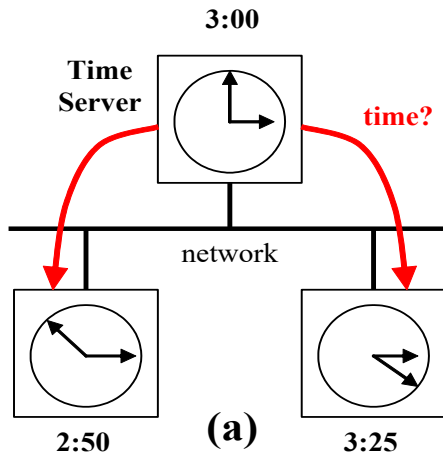
Berkeley Verfahren - Beispiel

- a) Server erfragt Zeit bei allen Clients
- b) Durchschnitt der Abweichungen $(-10+25+0) / 3 = 5$



Berkeley Verfahren - Beispiel

- a) Server erfragt Zeit bei allen Clients
- b) Durchschnitt der Abweichungen $(-10+25+0) / 3 = 5$
- c) Individuelle Korrekturwerte berechnen und versenden
 $5 - (-10) = +15$ $5 - 25 = -20$ $5 - 0 = +5$



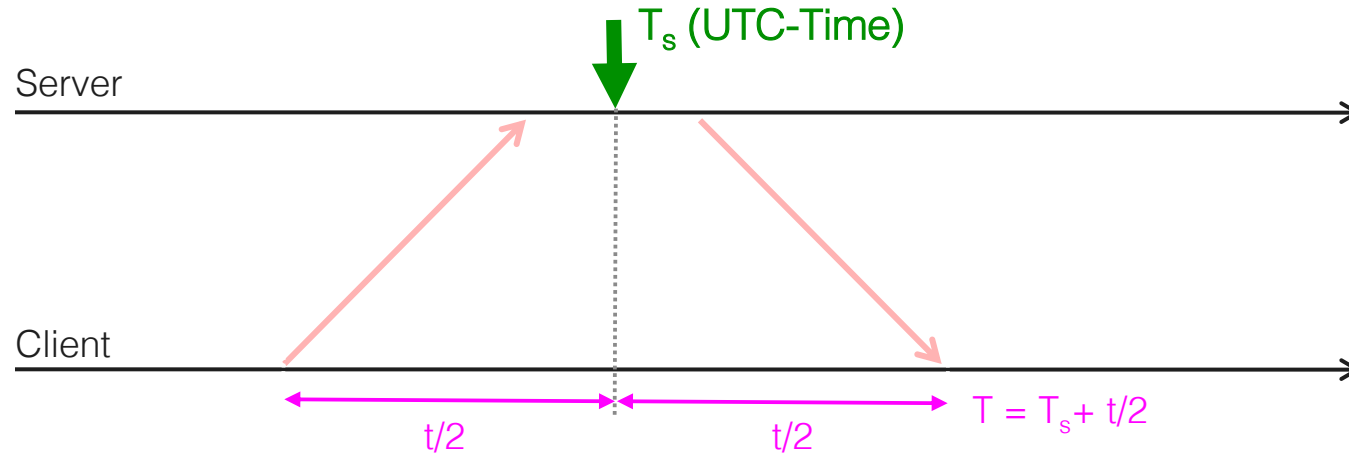
Synchronisierung realer Uhren - Cristian Verfahren

- Passiver zentraler Zeitserver, mit UTC-Empfänger
- Periodische Anfragen der Klienten an den Zeitserver
- Antwort des Zeitserver, eventuell mit Mittelwertbildung
- Halbieren der Antwortzeit ergibt ungefähre Zeit im Server

- Cristian F., "Probabilistic clock synchronization",
Distributed Computing, vol.3, no.3, 1989

Synchronisierung realer Uhren - Cristian Verfahren

- Ablauf:
 - Client misst Round-Trip-Time der Anfrage/Antwort
 - Client setzt seine Zeit beim Empfang der Antwort auf $T = T_s + t/2$



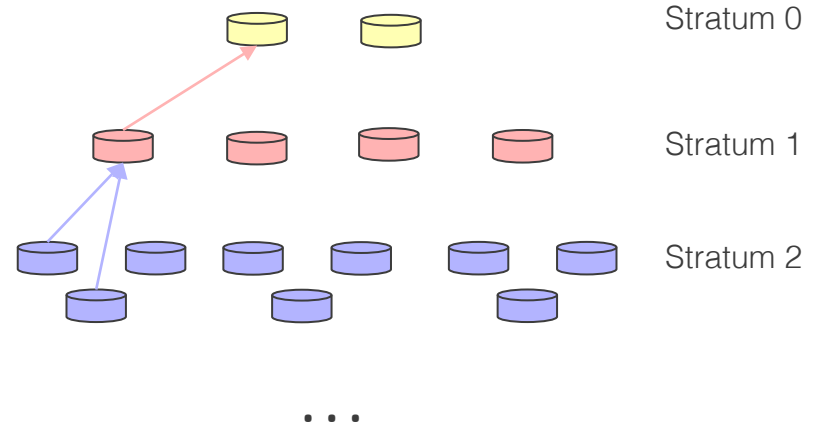
- Verbesserung: messe diesen Wert mehrfach und nehme den Durchschnitt, dabei „Ausreißer“ nicht berücksichtigen

Network Time Protocol (NTP)

- Ziel: weltweite Synchronisierung im Internet auf UTC
 - Auch über längere Zeiten ohne Verbindung
 - Kompensierung von typischer Clock-Drift
- Entwickelt in den 80er Jahren (RFC 1305)
 - Von Prof. David Mills
 - Siehe auch <http://www.eecis.udel.edu/~mills/ntp.html>
- UDP-basiertes Protokoll
- Logische Hierarchie der NTP-nutzenden Rechner →

Network Time Protocol - Stratum

- Stratum = Nummer der Ebene in der NTP-Hierarchie
 - Zeitgeber (Atomuhr, GPS-Empfänger, etc.) hat Stratum 0
 - Erster Rechner darunter hat damit Stratum 1
 - Definiert sind 16 Strata, i. d. R. werden nur vier genutzt



Network Time Protocol - Synchronisierung

- Synchronisierung der Server untereinander:
 - Server antwortet mit Zeitstempel (ähnlich Cristians Algorithmus)
 - Aber symmetrische Synchronisierung durch wechselseitigen Austausch von Zeitstempeln.
- Genauigkeit weltweit 1-50ms (siehe RFC 2030)
- NTP-Server:
 - Physikalisch Technische Bundesanstalt in Braunschweig (PTB):
`ptbtime1.ptb.de` und `ptbtime2.ptb.de` (Stratum 1)
 - Liste NTP-Server: <http://tf.nist.gov/tf-cgi/servers.cgi>

Steuerung der Uhrengeschwindigkeit

- Achtung: Zeit darf niemals rückwärts laufen
 - Uhr niemals zurücksetzen → Zeitstempel würden sich wiederholen
 - Sondern Geschwindigkeit graduell anpassen
- Systemzeit wird in Linux/MacOS/Microsoft Windows durch den Zeitgeber-Interrupt vorangetrieben
- Zeitgeber erzeugt z.B. alle 10ms einen Interrupt
 - Mit jedem Interrupt wird Systemuhr um $10 \pm \Delta t$ ms weitergeschaltet. Durch Anpassung von Δt kann die Geschwindigkeit genau eingestellt werden
 - Beispiel: Windows NT `SetSystemTimeAdjustment`

5.4 Lamport-Zeit

Logische-Zeit

- Es gibt verschiedene Verfahren um Uhren sehr genau zu synchronisieren, aber man schafft das nie exakt.
- Aber der genaue Zeitpunkt von Ereignissen ist meist nicht wichtig
- Oft reicht es die Abfolge von Ereignissen $e_1, e_2, \dots$ zu kennen
- Deswegen wurde die logische Zeit erfunden

Lamport-Zeit

- Ereignisse

- Resultieren aus dem lokalen Ereignisablauf auf einem Knoten sowie durch das Senden von Nachrichten zwischen Knoten
- Global gesehen wird eine Nachricht erst gesendet (Ursache: Sende-Ereignis) und dann empfangen (Wirkung: Empfangs-Ereignis)

- Um die Abfolge von Ereignissen $e_1, e_2, \dots$ zu definieren hat Lamport die "**Happened-Before**" Beziehung eingeführt

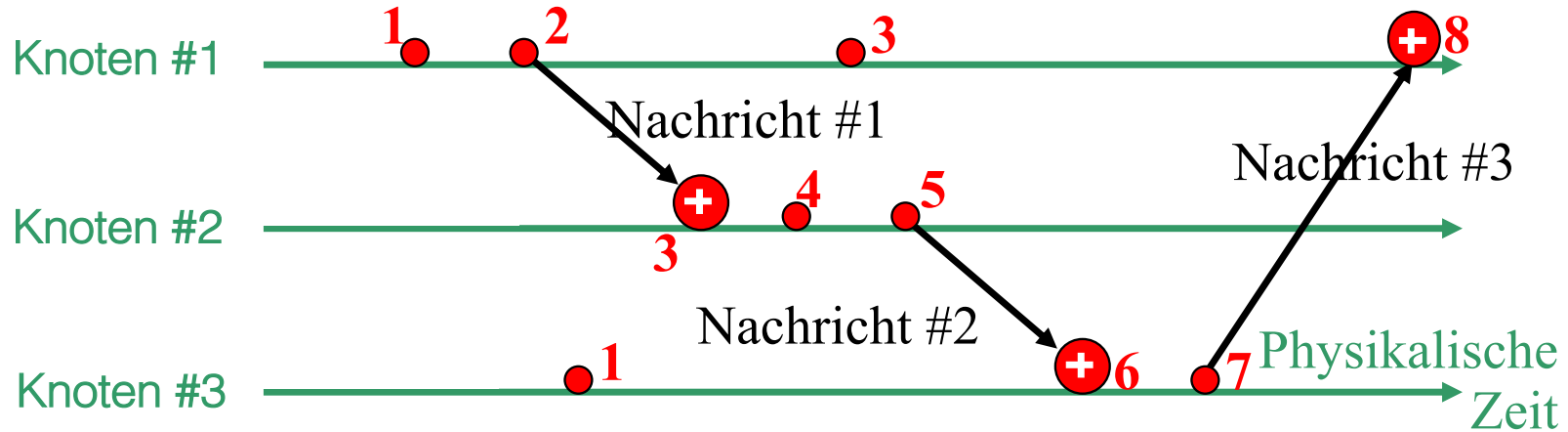
- Schreibweise: $e_1 \rightarrow e_2$
- Sprechweise: e_1 „happened before“ e_2
- Bedeutung: e_2 ist kausal abhängig von e_1
 - e_1 ist die Ursache, e_2 die Wirkung

Lamport-Zeit - Realisierung

- Rechner i verwaltet lokal seine Lamportzeit in einer Variablen TL_i
 - Datentyp: `uint64_t`
 - Wird mit 0 initialisiert
- Bei einem lokalen Ereignis wird die logische Zeit TL_i um eins erhöht
- Versendet Rechner i eine Nachricht, so wird TL_i inkrementiert und mit der Nachricht mitgeschickt
- Empfängt Rechner i eine Nachricht von Rechner j , so setzt der Empfänger i seine lokale Lamport-Zeit auf $TL_i = \max(TL_i, TS_j) + 1$

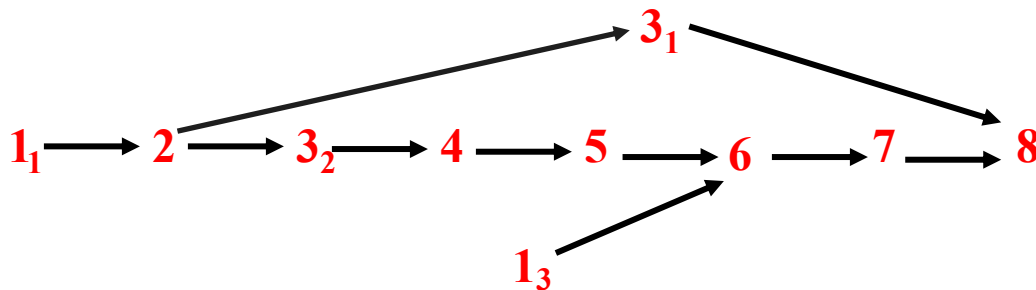
Lamport-Zeit - Beispiel

- Ereignis 3 auf #1 und Ereignis 4 auf #2 sind nebenläufig
→ relative Ordnung dieser Ereignisse ist algorithmisch irrelevant
- Resultate von (3,#1) oder (4,#2) werden (noch) nicht übertragen
→ eine kausale Beeinflussung ist in keine Richtung möglich



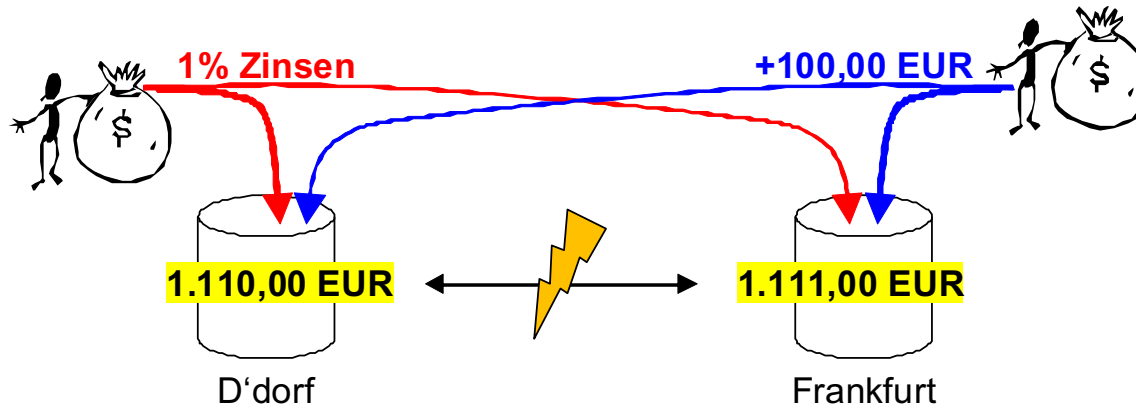
Lamport-Zeit - Uhrenbedingung

- Falls Ereignis b kausal von Ereignis a abhängt, dann muss der Zeitstempel von a kleiner als der Zeitstempel von b sein
- Es gilt: $e1 \rightarrow e2 \Rightarrow TL(e1) < TL(e2)$
- Achtung: $TL(e1) < TL(e2) \Rightarrow e1 \rightarrow e2$ gilt jedoch NICHT!
 - Im Beispiel: $TL(e3)_{\#1} < TL(e4)_{\#2}$, aber die Ereignisse sind nicht kausal abhängig
- Es entsteht eine Halbordnung (Beispiel von der Seite 29)



Lamport-Zeit - Beispiel: total geordneter Multicast

- Anwendungsbeispiel: zwei Kopien der selben Datenbank
 - Kontostand zu Beginn bei jeder Kopie 1.000,00 EUR
 - Kunde bezahlt in Frankfurt 100,00 EUR auf sein Konto ein
 - Gleichzeitig überweist die Bank in Düsseldorf 1% Zinsen auf den akt. Kontostand
 - Operationen werden per Multicast verschickt und kommen in unterschiedlicher Reihenfolge an



Lamport-Zeit - Beispiel: total geordneter Multicast (2)

- Ziel: konsistente Replikate → immer gleicher Kontostand
- Lösung: total geordneter Multicast
 - Alle Nachrichten werden in der selben Reihenfolge an alle Empfänger ausgeliefert
 - Annahme: zuverlässige Fifo-Streams zwischen jeweils zwei Knoten gegeben
 - Leicht realisierbar durch TCP-Sockets

Total geordneter Multicast: mit zentralem Sequenzer

- Implementierung-1: mit einem zentralen Sequenzer
 - Alle Sender müssen vor dem Senden eine Nummer dort anfordern.
 - Damit ist die Nummerierung und Sortierung der Nachrichten konsistent.
 - Und jeder Empfänger kann einfach erkennen, wenn eine Nachricht fehlt und diese lokal puffern und umsortieren.
-
- Pro: einfach zu implementieren
 - Contra: zentrale Komponente

Total geordneter Multicast: verteilte Lösung

- Implementierung-2: verteilte Lösung mithilfe der Lamport-Zeit
- **Zentrale Idee: Hold-Back Queue**
 - Nachrichten nicht sofort nach Empfang ausliefern, sondern puffern
 - Hierzu lokaler Puffer (pro Prozess) für eingehende Nachrichten
 - Nachrichten im Puffer sortieren nach Lamport-Zeitstempel des Senders

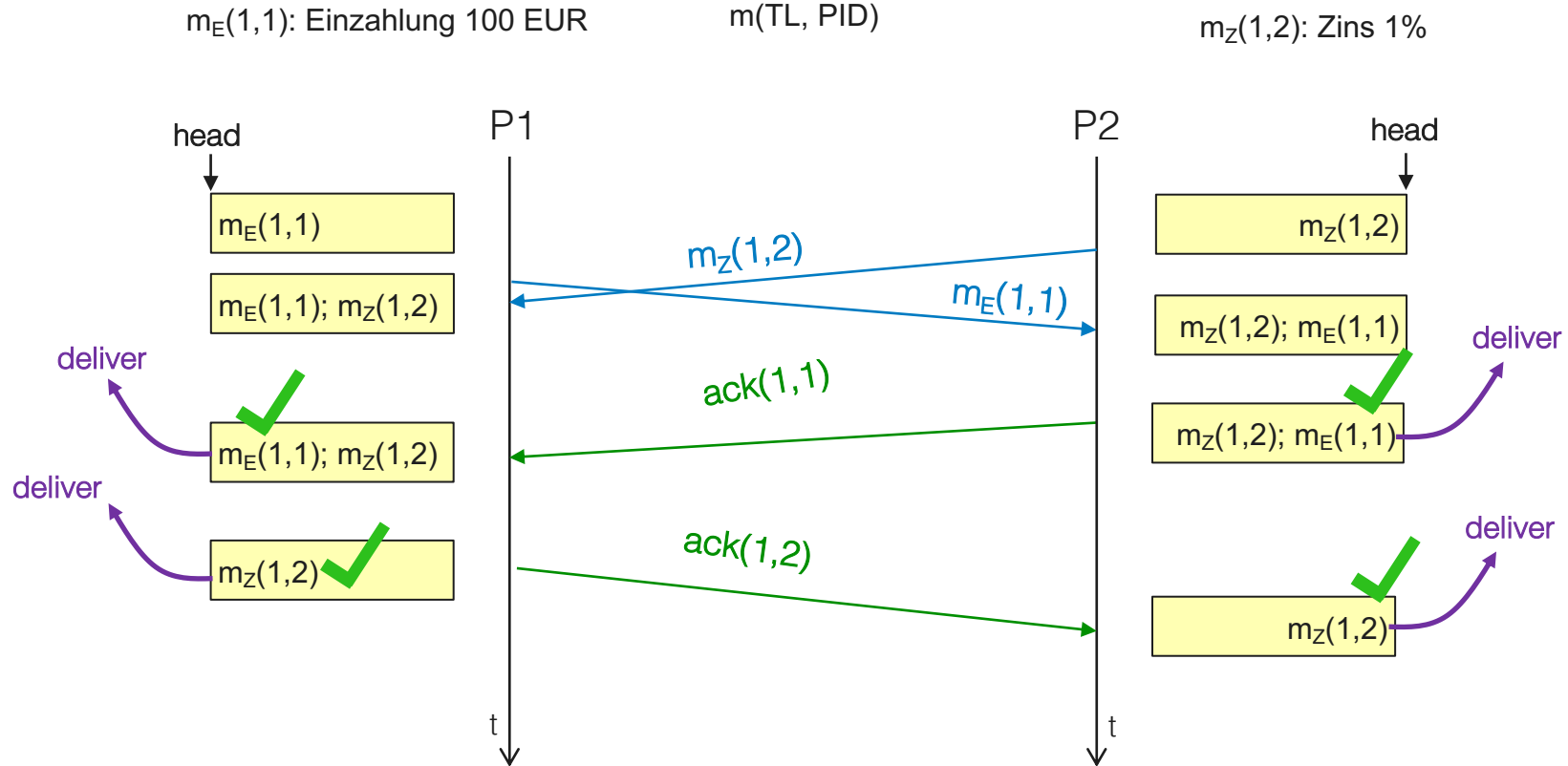
Total geordneter Multicast – verteilte Lösung

- Wir erweitern zunächst die Lamport-Zeit um die Prozess-ID
- Es gilt: $e1 \rightarrow e2$, falls
 - $TL_i(e1) < TL_j(e2)$ und zusätzlich
 - $TL_i(e1) == TL_j(e2)$ und $i < j$, wobei i und j Prozess-IDs sind
- Multicast-Nachricht m wird mit Lamport-Zeitstempel des Senders verschickt
- Empfangene Nachrichten werden in der Hold-Back-Queue nach dem Zeitstempel sortiert und per Multicast bestätigt
 - Durch die Erweiterung mit der Prozess-ID ist eine eindeutige Sortierung möglich

Total geordneter Multicast – verteilte Lösung

- Eine Nachricht in der Hold-Back-Queue wird ausgeliefert wenn:
 - Sie von allen anderen Prozessen bestätigt wurde
 - Und am Kopf der Queue ist
- Beobachtung: Irgendwann haben alle Prozesse dieselbe Kopie der Hold-Back Queue.
- Damit entsteht eine konsistente globale Reihenfolge für die Auslieferung der Nachrichten:
 - Zwei Multicast-Nachrichten haben nie den gleichen Zeitstempel wegen der Erweiterung durch die Prozess-ID

Total geordneter Multicast: Beispiel



5.5 Vektor-Zeit

- Friedemann Mattern, „Virtual Time and Global States of Distributed Systems“, in: Proceedings of the Workshop on Parallel and Distributed Algorithms, North-Holland, 1989.
- Vektorzeitstempel:
 - Wie bei Lamport Zeit gilt: $e1 \rightarrow e2 \Rightarrow VZ(e1) < VZ(e2)$
 - Aber nun gilt auch die Umkehrung: $VZ(e1) < VZ(e2) \Rightarrow e1 \rightarrow e2$
 - **Damit kann die Kausalität aus zwei Zeitstempeln abgeleitet werden**
- Vektorgröße: pro beteiligtem Knoten ein Vektorelement

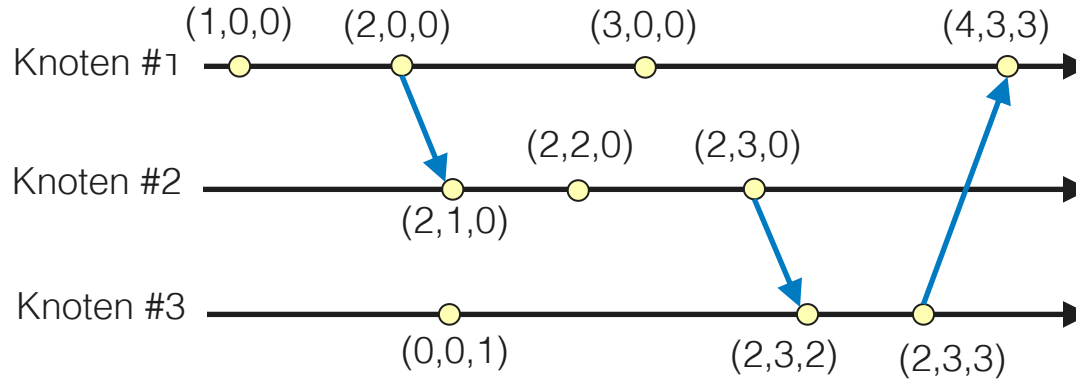
Vektor-Zeit - Aktualisierung des Vektors auf Knoten i

- Lokales Ereignis:
 $VZ[i] := VZ[i] + 1;$
- Nachricht senden:
 1. $VZ[i] := VZ[i] + 1;$
 2. VZ mitsenden,
- Nachricht empfangen:
 1. $VZ[i] := VZ[i] + 1;$
 2. $\forall j: VZ[j] = \max(VZ[j], VZ_{\text{empf}}[j]); j \neq i$

Vektor-Zeit - Vergleichsoperationen

- Vergleichsoperationen auf Vektorzeitstempel:
 - $VZ(e1) = VZ(e2)$, falls $\forall i: VZ(e1)[i] = VZ(e2)[i]$
 - $VZ(e1) < VZ(e2)$, falls $\forall i: VZ(e1)[i] \leq VZ(e2)[i]$
und $VZ(e1) \neq VZ(e2)$
- Bemerkungen:
 - Vektorzeitstempel $VZ(event)$ enthalten pro Knoten jeweils die höchste bekannte Zeit
 - u.U. erheblicher Speicherbedarf für Vektoren in den einzelnen Nachrichten, aber Komprimierung möglich.

Vektor-Zeit - Beispiel



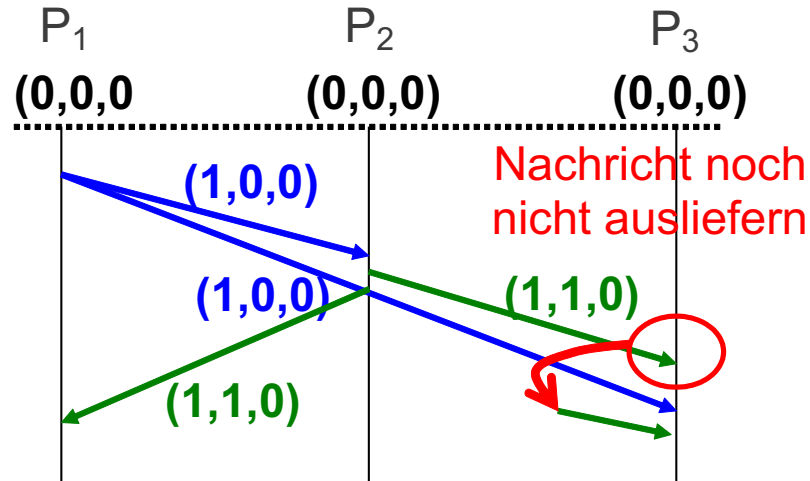
- $e1$ und $e2$ sind nebenläufig (" $\parallel$ ") falls $VZ(e1) \neq VZ(e2)$ und weder $VZ(e1) < VZ(e2)$ noch $VZ(e2) < VZ(e1)$:
 - Das heißt, Resultate von $e1$ können $e2$ nicht beeinflussen und umgekehrt
 - $(0,0,1) \parallel : (1,0,0), (2,0,0), (3,0,0), (2,1,0), (2,2,0), (2,3,0),$
 - $(3,0,0) \parallel : (2,1,0), (2,2,0), (2,3,0), (2,3,2).$

Vektor-Zeit - Anwendung: kausal geordneter Multicast

- Verwendet TCP-Verbindungen
- Initialisierung der Vektorzeitstempel: $\forall i: VZ[i]=0$
- P_s sendet Nachricht: $VZ_s[s] = VZ_s[s]+1$
- P_e empfängt Nachricht von P_s mit VZ_s
 - Zuerst Puffern bis gilt:
 - $VZ_s[s] = VZ_e[s]+1$ (dies ist die nächste Nachricht die P_e von P_s erwartet),
 - und $\forall k, k \neq s: VZ_s[k] \leq VZ_e[k]$ (diese Nachricht hängt kausal nicht von einer noch nicht empfangenen Nachricht ab)
 - Danach liefert P_e die Nachricht P_s mit VZ_s an die Anwendung aus
 - $VZ_e[s] = VZ_e[s]+1$

Vektor-Zeit - Anwendung: kausal geordneter Multicast

- Beispiel:

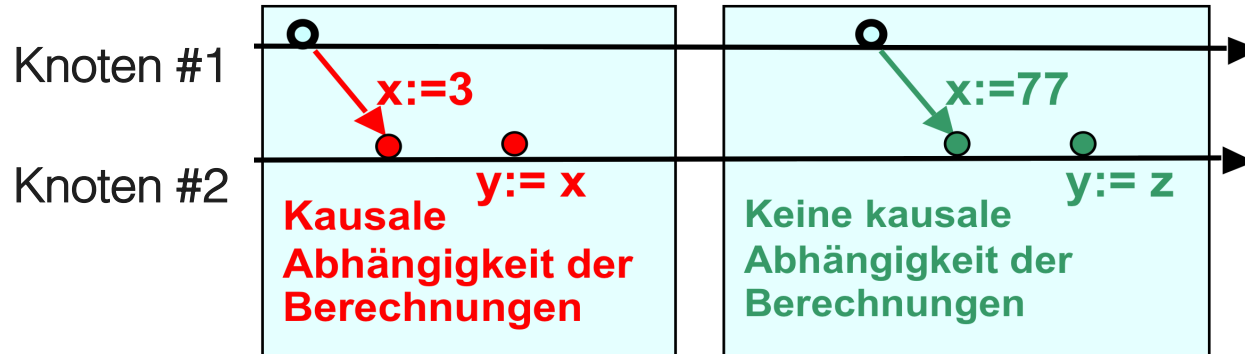


- A. Schiper, K. Birman, P. Stephenson, „Lightweight causal and atomic group multicast“, ACM Transactions on Computer Systems, Vol. 9, No. 3, August 1991.

5.6 Zusammenfassung

Kausalität

- Ohne Nachrichtenaustausch ist eine "ursächliche Verknüpfung" zwischen zwei oder mehr Knoten unmöglich (keine Kausalität).
- Das Resultat einer Berechnung kann die Berechnung in einem zweiten Knoten nur über eine Nachricht beeinflussen.
- Aber beim Nachrichtenaustausch ist Kausalität möglich, aber nicht zwingend:



Zusammenfassung

- Uhren können synchronisiert werden, aber nicht beliebig genau
- Häufig reicht es die Reihenfolge von Ereignissen in einem verteilten System zu kennen, wofür eine logische Zeit reicht
- Die Lamport-Zeit ist einfach zu implementieren, erlaubt aber keinen Rückschluss auf kausale Abhängigkeiten anhand der Zeitstempel
- Letzteres ermöglicht die Vektorzeit, welche jedoch aufwändig ist, da sie ein Element pro beteiligtem Rechner benötigt